

Replayable ALPHA Betting-Martingale Audit Transcripts

Gio Della-Libera

Draft — June 28, 2026

Abstract

ALPHA treats a risk-limiting audit as a test-supermartingale for a bounded-mean assertion. Rather than trusting a reported risk value, RCOUNT replays the published assorter values and betting parameters, recomputes the nonnegative martingale in exact rational arithmetic, converts the running statistic to a parts-per-million p -value, and checks the declared stop-or-continue decision. This paper documents the implemented V.15 slice: a single-assertion, with-replacement, fixed-bet bounded-mean martingale exposed through `alpha-martingale-v1`. The claim is narrow but useful: when the public audit record declares the assorter, upper bound, risk limit, ordered sample values, and per-step bets, an independent verifier can regenerate the ALPHA transcript from package evidence.

1 Introduction

ALPHA and related betting-martingale audits are usually presented as a flexible statistical framework: encode an election assertion as a bounded mean, observe a sample stream, choose a betting parameter, and update a test statistic until the risk limit is met. The public-verification question is lower level. An observer must be able to ask whether the declared martingale transcript follows from the declared assorter values and bets.

RCOUNT therefore treats ALPHA as a replay contract. The verifier does not trust a reported martingale value or a vendor-exported p -value. It reads the audit record, reconstructs the bounded-mean observations, and recomputes the nonnegative martingale. Under the null, that martingale never grows in expectation; large realized values are evidence against the null, and the running inverse statistic supplies the replayed p -value.

This paper covers the implemented V.15 slice. It does not claim to implement every ALPHA audit design, adaptive bet selector, or sampling mode. It claims that the current fixed-bet bounded-mean transcript is reproducible when the package records the assertion, sample order, assorter values, risk limit, and bet sequence.

2 ALPHA Contract

The audit record carries the public inputs the ALPHA replay needs:

Field	Meaning
<code>method_id</code>	Versioned method name. Current value: <code>alpha-martingale-v1</code> .
<code>risk_limit_ppm</code>	Risk limit in parts per million. The stop threshold is $1/\alpha$.
<code>sampling_mode</code>	Declared sampling mode, currently recorded as package evidence while the implemented replay exercises the fixed-bet bounded-mean path.
<code>assertions</code>	The implemented replay requires exactly one bounded-mean assertion.
<code>assertion_id</code>	Identifier binding each sample step to the assertion under test.
<code>assorter_id</code>	Identifier for the function that maps sampled evidence to a bounded value.
<code>assorter_upper_bound</code>	Upper bound u for every declared assorter value.
<code>sample_steps</code>	Ordered sample stream.
<code>assorter_value</code>	Rational observation x_i for a sample step.
<code>bet</code>	Rational betting parameter λ_i used at that step.
<code>statistic</code>	Optional declared martingale value to check against replay.
<code>p_value_ppm</code>	Optional declared p -value in parts per million to check against replay.

The implemented null mean is half the declared assorter upper bound, $m_0 = u/2$. Starting from $M_0 = 1$, RCOUNT replays each step by exact rational arithmetic:

$$F_i = 1 + \lambda_i \frac{x_i - m_0}{u}, \quad M_i = M_{i-1} F_i.$$

The update factor must not be negative, and each x_i must lie in $[0, u]$. The replayed p -value is the ceiling, in integer parts per million, of $1/M_i$. The computed decision is `pass` once $M_i \geq 1/\alpha$ and `continue` otherwise.

3 Implemented Surface

The algorithm-replay dispatcher in `rcount-audit` routes `ALPHA_MARTINGALE_METHOD_ID` to `replay_alpha_run`. That function first enforces the public contract: `risk_limit_ppm` must be present, exactly one assertion must be declared, the assertion upper bound must be a valid rational value, and every sample step must carry a `bet`. Missing bets produce a boundary transcript rather than a statistical verdict.

When those fields are present, `replay_alpha_run` converts each sample step to a `BoundedMeanMartingale` and calls `replay_fixed_bet_bounded_mean_martingale` in `rcount-stats`. The statistics crate owns the bounded-mean recurrence, exact rational update factors, running martingale, integer-ppm p -value conversion, and stop threshold. The audit crate then compares the computed statistic, p -value, and stop flag with any declared per-step values and reports drift through the common algorithm transcript.

Check	Failure caught
<code>audit_algorithm_run</code>	Unsupported method id, missing ALPHA risk limit, multiple or absent assertions, invalid assertion bound, or missing per-step bets.
<code>alpha_replay</code>	Declared martingale statistic, p -value, or stop decision drifts from the recomputed bounded-mean transcript.
<code>bounded_mean_martingale</code>	Assorter values outside the declared range, invalid null mean or upper bound, invalid risk limit, or a negative martingale update factor.
<code>cvr_summary_total</code>	Public summaries still reconcile before audit-algorithm replay is interpreted with the rest of the package.
<code>source_hash_match</code>	Source evidence bytes still match the declared source index.

This is a fixed-bet surface. The bet can vary by step because it is recorded in the transcript, but the current code replays the declared sequence; it does not choose an adaptive betting strategy for the verifier.

4 Fixtures

The package-level positive fixture is exercised by `replay_audit_algorithms_outputs_alpha_transcript`. It builds an audit run named `audit-run:alpha-toy` with method `alpha-martingale-v1`, one assorter-mean assertion, four sample steps with assorter value 1, four declared bets of 1, and a 250000 ppm risk limit. The CLI replay reports `status pass`, computed decision `pass`, final statistic 81/16, and final p -value 197531 ppm.

The package-level drift fixture is `replay_audit_algorithms_exits_one_on_alpha_declared_drift`. It uses the same toy run shape under `audit-run:alpha-drift`, but edits the declared final p -value to 999999 ppm. The recomputed transcript disagrees and the command exits with failure while exposing the declared p -value mismatch.

The audit-crate drift unit test is `alpha_algorithm_statistics_replay_detects_declared_drift`. It constructs `audit-run:alpha-toy` directly and checks that the fourth step carries a `p-value mismatch` error when the declared value is edited.

The boundary fixture is `replay_audit_algorithms_outputs_alpha_boundary_when_bets_are_missing`. It names `audit-run:alpha-boundary` and omits the per-step bet. The package remains readable, but ALPHA replay returns a boundary transcript with the message `ALPHA replay requires bet on every sample step`. The matching audit-crate unit test is `alpha_algorithm_statistics_replay_requires_bet`.

The numerical positive fixture is `fixed_bet_bounded_mean_martingale_replays_exactly` in `rcount-stats`. It checks the shared primitive directly: null mean 1/2, upper bound 1, risk limit 250000 ppm, four observations of value 1 and bet 1, stop threshold 4, final martingale 81/16, and final p -value 197531 ppm. The numerical boundary fixture `fixed_bet_bounded_mean_martingale_rejects_out_of_range_values` confirms that an observation outside the declared bound is rejected.

There is not yet a named ALPHA continue fixture in the implementation. The plan also leaves adaptive-bet and without-replacement fixtures unchecked, so this paper does not describe them as implemented evidence.

5 Boundaries

The claim boundary is the central V.15 point: a martingale value alone is not an independently replayable audit claim. The transcript must record the betting rule or, in the current implementation, the actual bet sequence. It must also record the assorter identity, the assertion upper bound, the sampling mode, and the draw order. Without those fields, RCOUNT can preserve the evidence but cannot regenerate the statistical claim.

The implemented recurrence is a bounded-mean betting martingale. It is not a BRAVO ballot-polling likelihood ratio, a Minerva or Athena round calculation, a Kaplan-Markov comparison audit, a batch-comparison taint product, or a stratified combined-risk calculation. Those methods have separate method ids or separate boundary records in the audit replay dispatcher.

The bet-selection boundary is equally important. ALPHA can support adaptive betting strategies, but the current replay does not infer such a strategy from the observations. It verifies the declared transcript: each per-step bet must be present, and the verifier recomputes what follows from that bet. A future adaptive fixture must therefore declare enough deterministic bet-selection state for the same replay principle to hold.

Finally, this slice does not prove sample selection, ballot custody, CVR interpretation, or legal compliance. It assumes the ordered sample stream and assorter values are the public evidence being checked by this method. Sampler replay, summary reconciliation, and source hashing remain separate RCOUNT contracts.

6 Conclusion

RCOUNT now has a replayable first ALPHA surface. The verifier reads a single bounded-mean assertion, its upper bound, a risk limit, ordered assorter values, and declared bets; then it recomputes the fixed-bet martingale, integer-ppm p -values, and pass-or-continue decision through the shared `rcount-stats` primitive. Missing bets and invalid numerical inputs produce explicit boundary or failure evidence rather than a trusted statistical verdict.

The implementation is deliberately smaller than the full ALPHA design space. It establishes the transcript discipline that later adaptive, without-replacement, stratified, and SHANGRLA-style work will need: the public record must contain enough information to replay the bet/statistic path, not merely the final risk number.

References

- Gio Della-Libera. Rcount overview: Reproducible election count packages. Technical report, Apportionment Research, 2026.
- Philip B. Stark. Alpha: Audit that learns from previously hand-audited ballots. Technical report, arXiv, 2022.